

# Reinforcement Learning

Lecture Notes

Carlos Cotrini  
ETH Zürich

June 18, 2026

These notes accompany the interactive visualization *Repair or Replace*, available at

<https://eth-aint-fs26.github.io/rl-arcade/repair-or-replace/>

The concepts developed below map one-to-one onto its scenes.

## 1 Motivation

Many decisions a manager faces are sequential. What is decided today changes what must be decided tomorrow, and the consequences unfold over weeks and months rather than at once. The outcomes are uncertain on top: the same decision may turn out well one time and badly the next. Picture the scenario behind these notes. You operate a delivery business with a single van, and each week you make exactly one call about that machine: keep running it and collect the week's profit, at the risk of a breakdown and of gradual wear; repair it, paying for a week in the shop so that it fails less often afterwards; or replace it outright, a large expense that buys a fresh start. Every option trades a short-term cost against a long-term payoff, and chance interferes at every step.

Reinforcement learning is a framework for computing decision strategies that maximize the reward accumulated over time in precisely such problems. One running example, the repair-or-replace problem of the visualization, carries everything.

**The plan.** These notes answer one practical question by climbing a short ladder of smaller ones. How do we pick the best call? The one with the largest long-run reward. While we are still learning, how do we split our weeks between the call we already believe is best and others we have yet to measure? That is the explore-exploit tension. How do we know a call's reward in the first place? The Bellman equation gives it, as a number  $Q^*(s, a)$  attached to every state and call. And when the odds are hidden, as they always are in real life? Then that reward is an average we cannot compute but can only sample, one lived week at a time; a result of Robbins and Monro tells us how to learn such an average from its samples, and SARSA applies that rule to the Bellman number. Sections 5, 6, 7, and 8 are the rungs. The interactive [visualization](#) draws this same ladder as a row of rides, the MDP maze, the bandit arcade, Bellman's house, darts in the dark, and the SARSA maze; if a picture helps, that is the order below.

## 2 Markov decision processes

We start with the first rung: what exactly are we deciding, and against what? The answer is a precise model of the decision problem, and the standard model is the *Markov decision process*.<sup>1</sup>

**Definition 2.1** (Markov decision process). A *Markov decision process* (MDP) is a quadruple, a package of four ingredients,  $(S, A, T, \gamma)$ , where  $S$  is the set of *states* the system can be in;  $A$  is the set of *actions* available to the decision maker;  $T$  is the *transition dynamics*, which describes how the system reacts when an action is taken; and  $\gamma \in [0, 1]$ , the *discount factor*, is a number between 0 and 1. ( $\gamma$  is the Greek letter gamma.)

The sets  $S$  and  $A$  are defined a priori: they are fixed in advance, part of the problem description, and writing them down is the modeler’s first task. The transition dynamics is then defined on top of them. It is a *randomized function*  $T$  that consumes a state  $s$  and an action  $a$  and produces a new state  $s'$  and a reward  $r$ :

$$(s', r) = T(s, a).$$

In words: feed the current state and the chosen action into the dynamics, and out come the state the system lands in next and the reward, a number measuring the immediate gain or loss of that step.

*Remark 2.2.* The transition dynamics is *random*: feeding the same state and action  $(s, a)$  in twice may produce different new states and different rewards.<sup>2</sup>

The discount factor  $\gamma$  is just a number between 0 and 1; its use will be explained later.

**Example 2.3** (the repair-or-replace MDP). You manage a one-van delivery fleet; the visualization calls the van *Old Bessie*. Each week the van sits at one of four wear levels, and you make exactly one call:

$$S = \{\text{HEALTHY, WORN, SHAKY, FAILING}\}, \quad A = \{\text{RUN, SERVICE, REPLACE}\}.$$

In words: four wear levels, from factory-fresh to nearly dead, and three possible calls (SERVICE is the “repair” of the title). Rewards are weekly net money, in plain units. The numbers governing RUN are:

| state   | weekly profit | breakdown odds |
|---------|---------------|----------------|
| HEALTHY | +95           | 2%             |
| WORN    | +72           | 8%             |
| SHAKY   | +40           | 28%            |
| FAILING | +16           | 55%            |

Under RUN the van drives the route and earns the profit in the table, unless it breaks down, with the odds in the table: the week then books a single net reward equal to the profit minus a breakdown cost of 280 (from HEALTHY:  $95 - 280 = -185$ ), and the van ends the week at FAILING. Even without a breakdown, wear tends to slip by one or two levels over the weeks. Under SERVICE the van spends the week in the shop: reward  $-50$ , no deliveries, and the wear often improves; from WORN, the van returns to HEALTHY 95% of the time. Under REPLACE the reward is  $-130$ , and the next week starts with a brand-new van at HEALTHY, whatever the old one looked like. The dynamics is random in the sense of Remark 2.2: feeding (WORN, RUN) into  $T$  may return (SHAKY, +72) one week and (FAILING,  $-208$ ) another. There is no final week (the fleet runs indefinitely), and the visualization fixes  $\gamma = 0.9$ .

<sup>1</sup>The name records that the next state depends only on the current state and action, not on the past.

<sup>2</sup>Scene “You run the fleet” of the interactive [visualization](#) lets you sample these dynamics.

### 3 Policies and trajectories

An MDP describes the world the manager operates in; it does not say how to operate. That is the job of a policy.

**Definition 3.1** (policy). A *policy* defines how to act at each state. Formally, a policy is a function  $\pi: S \rightarrow A$ : it takes a state  $s$  and returns the action  $\pi(s)$  to be played whenever the system is in  $s$ . ( $\pi$  is the Greek letter pi.)

Executing a policy on the MDP, week after week, generates a record of everything that happened. That record is the central object of these notes.

**Definition 3.2** (trajectory). The *trajectory* of a policy  $\pi$  is the sequence

$$s_1, a_1, r_1, s_2, a_2, r_2, \dots, s_i, a_i, r_i, \dots$$

produced by executing  $\pi$  on the MDP starting from state  $s_1$ . In words: states, actions, and rewards alternate, and the subscript counts the weeks. The mechanics are as follows. The first action results from applying the policy to the first state,  $a_1 = \pi(s_1)$ . The first reward and the second state result from applying the transition dynamics to that pair,  $(s_2, r_1) = T(s_1, a_1)$ . Then  $a_2 = \pi(s_2)$ , the dynamics produce  $(s_3, r_2) = T(s_2, a_2)$ , and so on without end.

*Remark 3.3.* The trajectory is *random*: starting again from the same  $s_1$  and executing the same policy may produce a different trajectory, because the transition dynamics is random (and so, possibly, is the policy, if one allows policies that randomize their choices).

**Example 3.4** (a trajectory of the fleet). Consider the playbook  $\pi$  with  $\pi(\text{HEALTHY}) = \text{RUN}$ ,  $\pi(\text{WORN}) = \text{SERVICE}$ , and  $\pi(\text{SHAKY}) = \pi(\text{FAILING}) = \text{REPLACE}$ . One execution of  $\pi$  from  $s_1 = \text{HEALTHY}$  produced<sup>3</sup>

|                         |                         |              |
|-------------------------|-------------------------|--------------|
| $s_1 = \text{HEALTHY},$ | $a_1 = \text{RUN},$     | $r_1 = +95,$ |
| $s_2 = \text{WORN},$    | $a_2 = \text{SERVICE},$ | $r_2 = -50,$ |
| $s_3 = \text{HEALTHY},$ | $a_3 = \text{RUN},$     | $r_3 = +95,$ |
| $s_4 = \text{HEALTHY},$ | $a_4 = \text{RUN},$     | $r_4 = +95,$ |

after which  $s_5 = \text{WORN}$  and the walk continues, one week per step, forever. In words: the van started healthy and earned 95 but slipped to worn; a 50 service restored it; two profitable runs followed; by week five it was worn again. Remark 3.3 is at work here: another execution of the same  $\pi$  from the same  $s_1$  might open with a first-week breakdown,  $s_1 = \text{HEALTHY}$ ,  $a_1 = \text{RUN}$ ,  $r_1 = -185$ ,  $s_2 = \text{FAILING}$ .

### 4 The goal: accumulating reward

Which call wins in the long run? To answer that we first need a way to score a whole tape: a policy is good if it produces trajectories that accumulate as much reward as possible. The first task is to make “accumulate” precise.

**Definition 4.1** (cumulative reward). Take one particular trajectory  $s_1, a_1, r_1, s_2, a_2, r_2, \dots$ . Its *cumulative reward*, also called its *return*, is

$$G_1 = r_1 + \gamma r_2 + \gamma^2 r_3 + \dots + \gamma^{i-1} r_i + \dots \quad (4.1)$$

In words: add up the rewards of all weeks, forever (that is what the dots mean), scaling the reward of week  $i$  by  $\gamma^{i-1}$ : week one counts in full, week two is scaled by  $\gamma$ , week three by  $\gamma^2$ , and so on. ( $G$  stands for *gain*.)

---

<sup>3</sup>Scene “The trajectory” of the interactive [visualization](#) replays such a tape week by week.

Here the discount factor plays its very important role: it specifies how much we care about future rewards. With  $\gamma$  close to 0, the scaling factors die out almost immediately and only near-term rewards matter; with  $\gamma$  close to 1, they fade slowly and rewards far in the future count almost as much as immediate ones. Managers will recognize the device: it is the discounting of future cash flows familiar from net-present-value calculations, and, as with the NPV of a perpetuity, the shrinking factors keep the never-ending total finite. With the visualization's  $\gamma = 0.9$ , next week's reward counts 90% of this week's, the week after 81%, and so on.<sup>4</sup> For the trajectory of Example 3.4, formula (4.1) gives  $G_1 = 95 + 0.9 \cdot (-50) + 0.9^2 \cdot 95 + 0.9^3 \cdot 95 + \dots = 95 - 45 + 76.95 + 69.255 + \dots$ .

The same accounting can be started at any week, not only the first. For an arbitrary week  $i$ , define  $G_i$  as the cumulative reward from week  $i$  on:

$$G_i = r_i + \gamma r_{i+1} + \gamma^2 r_{i+2} + \dots \quad (4.2)$$

In words:  $G_i$  is what the trajectory accumulates from week  $i$  onward, with the discounting restarted there. Comparing (4.2) at weeks  $i$  and  $i + 1$  yields a recurrence that carries the rest of these notes:

$$G_i = r_i + \gamma G_{i+1}. \quad (4.3)$$

In words: what is accumulated from week  $i$  on equals this week's reward plus  $\gamma$  times what is accumulated from week  $i + 1$  on.

*Remark 4.2.*  $G_i$  is *random*. Because of the randomness of the transition dynamics,  $G_i$  comes out differently after every execution of the policy starting from state  $s_i$ .

A quantity that changes from run to run cannot be maximized directly; we average it instead.

**Definition 4.3** (expectation). The *expectation*  $\mathbb{E}[G_i]$ , read “the expected value of  $G_i$ ,” is the average of  $G_i$  over a sufficiently large set of trajectories: execute the policy many times, always starting from the same first state  $s_1$  (for the fleet: a HEALTHY van), record the value of  $G_i$  of each execution, and average the records.

We can now state the ultimate objective of reinforcement learning: *find the policy  $\pi$  that maximizes  $\mathbb{E}[G_1]$* , the policy whose trajectories accumulate, on average, the most reward. A policy achieving this maximum is called *optimal*. Note the contrast with  $G_i$  itself:  $\mathbb{E}[G_i]$  is a single real number that does not depend on the trajectory, because the averaging has removed the randomness. Therefore  $\mathbb{E}[G_i]$  is *not* random.

## 5 Exploration and exploitation

Before we can compute the best call we have to live through many weeks, and in each of those weeks we must actually pick a call. That raises a tension with nothing to do with vans in particular and everything to do with learning under uncertainty. You can *exploit*: take the call you currently believe is best and bank the reward you expect from it. Or you can *explore*: take a different call, not because you think it is better but because you want to find out what it is worth. The two pull against each other, and you cannot do both in the same week.

Why not simply always exploit? Because you cannot pile up reward on a call you have never measured. A call that is never tried is never learned: you will never discover what servicing a worn van is really worth if you never once service it. Pure exploitation locks in whatever guesses you happened to start with; pure exploration never cashes in on what it learns. A good learner does mostly the first and a little of the second.

---

<sup>4</sup>Scene “Return over the van’s life” of the interactive [visualization](#) has a slider for  $\gamma$ .

**Definition 5.1** (the  $\varepsilon$ -greedy rule). Fix a small number  $\varepsilon$  between 0 and 1 (the visualization uses  $\varepsilon = 0.15$ ;  $\varepsilon$  is the Greek letter epsilon). At each state, with probability  $1 - \varepsilon$  take the call currently believed best (exploit), and with probability  $\varepsilon$  take a call drawn at random instead (explore).

The single knob  $\varepsilon$  sets the balance: at  $\varepsilon = 0$  the rule never explores, at  $\varepsilon = 1$  it never exploits, and a small positive value like 0.15 spends most weeks earning while still, now and then, sampling a call it would otherwise neglect.<sup>5</sup> We will need exactly this rule in Section 8: SARSA learns while it drives, so every week it must pick a call, and it picks by the  $\varepsilon$ -greedy rule.

## 6 The $Q$ -function and the Bellman equation

So: which call wins in the long run, and how do we know its value? The classical route goes through an auxiliary function that scores every pair of a state and an action, and the Bellman equation pins that score down.

**Definition 6.1** (the  $Q$ -function). For a state  $s$  and an action  $a$ ,  $Q^*(s, a)$  is the cumulative reward one can expect when executing an optimal policy from state  $s$ , except that the *first* action taken there is forced to be  $a$ . In words: average  $G_i$  over many executions, keeping only the executions that are in state  $s$  at week  $i$  and play action  $a$  there as a one-off, even when an optimal policy would choose differently, and act optimally on every later week; that average is  $Q^*(s, a)$ . (The star in  $Q^*$  marks optimality.)

Which week  $i$ ? It does not matter: the fleet has no final week, so it looks the same from week 3 as from week 30, and every week gives the same average.

*Remark 6.2.*  $Q^*(s, a)$  is different from  $\mathbb{E}[G_i]$ . The number  $\mathbb{E}[G_i]$  averages over *all* executions of the policy, whatever state they happen to be in at week  $i$ .  $Q^*(s, a)$  restricts the average to the executions that pass through  $s$  at week  $i$  and are made to play  $a$  there, acting optimally afterwards. And while  $\mathbb{E}[G_i]$  is a single number,  $Q^*$  is a whole table of numbers, one per state-action pair: for the fleet,  $4 \times 3 = 12$  entries.

The table is worth the trouble because the optimal policy can be read off it: at state  $s$ , the optimal action is the action  $a$  for which  $Q^*(s, a)$  is largest. We write  $\pi^*(s)$  for this action, and  $\pi^*$  for the resulting optimal policy.

It remains to compute the table. Suppose an optimal policy produced the trajectory  $\dots, s_i, a_i, r_i, s_{i+1}, a_{i+1}, \dots$ . Averaging both sides of the recurrence (4.3),  $G_i = r_i + \gamma G_{i+1}$ , over many such executions, it must be the case that the *Bellman equation* holds:

$$Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]. \quad (6.1)$$

In words: the value of this week's state and action equals this week's reward plus  $\gamma$  times the value of next week's state and action, averaged over the randomness of the week. Replacing  $G_{i+1}$  by its average  $Q^*(s_{i+1}, a_{i+1})$  is legitimate: averaging in two rounds (first among the executions that share next week's state and action, then across the week's outcomes) gives the same result, just as a company-wide average can be computed branch by branch.

We can therefore find  $Q^*$  by solving the Bellman equations, one equation per state-action pair, twelve for the fleet. Since the optimal policy at  $s_{i+1}$  picks the action with the largest  $Q^*$ -value, the right-hand side of (6.1) involves the same unknowns  $Q^*(s, a)$  as the left-hand side, so this is a system of equations in the entries of the table. The visualization solves it

<sup>5</sup>This is the same explore-exploit dial met for the multi-armed bandit; scene “the bandit arcade” of the interactive [visualization](#) is its playground.

iteratively; its  $Q^*$ -scorecard and Bellman-check scenes display the solution for the fleet at  $\gamma = 0.9$ .<sup>6</sup> The full scorecard for the fleet at  $\gamma = 0.9$  is

|         | RUN          | SERVICE      | REPLACE      |
|---------|--------------|--------------|--------------|
| HEALTHY | <b>311.0</b> | 229.9        | 149.9        |
| WORN    | 203.4        | <b>226.1</b> | 149.9        |
| SHAKY   | 96.5         | 126.4        | <b>149.9</b> |
| FAILING | -3.1         | 86.9         | <b>149.9</b> |

where the largest value in each row (in bold) marks the optimal call there. Reading down those crowned cells,  $\pi^*$  is: RUN when HEALTHY, SERVICE when WORN, REPLACE when SHAKY or FAILING. At SHAKY, REPLACE at 149.9 edges out SERVICE at 126.4, so the optimal playbook replaces a van that still starts every morning. Notice too that REPLACE scores 149.9 in *every* row: replacing always lands a brand-new HEALTHY van next week, so its value cannot depend on the wreck being scrapped.

## 7 Learning a number in the dark: Robbins-Monro

The Bellman equation hands us the scorecard, but only if we know the odds: its right-hand side is an average over the randomness of the week, and to average over outcomes one must know how likely each outcome is. When the odds are hidden, that average is exactly what we cannot compute. So strip the fleet away and isolate the bare problem underneath: find the number  $x^*$  that *equals the average* of some noisy evidence, when the average cannot be computed directly and the evidence arrives one sample at a time. The game “darts in the dark” of the interactive [visualization](#) is its picture: you never see the bullseye, only where each throw lands, and you must home in on the average landing point.

The *Robbins-Monro algorithm* keeps a running guess  $x$  and, when the  $n$ -th sample arrives, nudges it a fraction of the way toward that sample:

$$x \leftarrow x + \alpha_n (\text{sample} - x). \quad (7.1)$$

In words: move the guess a fraction  $\alpha_n$  of the way toward each sample as it arrives. The bracketed sample  $- x$  is a *surprise*, the gap between the newcomer and the current guess. The step sizes  $\alpha_1, \alpha_2, \alpha_3, \dots$  must meet two demands. The nudges must add up without bound, meaning that  $\alpha_1 + \alpha_2 + \alpha_3 + \dots$  grows past any number, so that the guess can travel from any bad start all the way to  $x^*$ . And their squares must add up to something finite, meaning that  $\alpha_1^2 + \alpha_2^2 + \alpha_3^2 + \dots$  stays bounded, so that the noise in the samples eventually cancels out instead of jiggling the guess forever. The choice  $\alpha_n = 1/n$  meets both: the running total  $1 + 1/2 + 1/3 + \dots$  climbs past any bound, just ever more slowly (a classical, if surprising, fact), while the squared total  $1 + 1/4 + 1/9 + \dots$  settles below 2. The recipe is due to Herbert Robbins and Sutton Monro, who published it in 1951; Appendix A gives an elementary, two-line derivation of the same nudge.

This is the entire engine SARSA runs on. The next section applies the very nudge in (7.1), cell by cell, with the arriving “sample” being one lived week’s evidence about the value of a call. One honest wrinkle will appear there: SARSA measures that sample using the same table it is updating, so the algorithm pulls itself up by its own bootstraps, and making this fully rigorous took decades after 1951.

<sup>6</sup>Scenes “ $Q^*$ : the action scorecard” and “The Bellman equation,” interactive in the [visualization](#).

## 8 SARSA: learning from the logbook

This is the last rung, and it ties the previous two together. Section 7 showed how to learn an average from its samples; the Bellman equation (6.1) says the number we want *is* such an average, over the randomness of the week. Put the two side by side and a plan appears: rather than compute the Bellman average, which needs the odds nobody hands us (no manual lists the breakdown odds for *your* van on *your* routes), *sample* it, one lived week at a time, and feed those samples to the Robbins-Monro nudge. The odds never have to be written down. And they had better not have to be: for realistic fleets the table of states explodes anyway (count odometer readings, age, weeks since the last service, route load, then forty vans, and the visualization’s illustrative tally reaches some four million states for a single van and half a billion scorecard cells for a modest fleet).<sup>7</sup> What the manager does have is the *logbook*, week after week of lived experience: the state the van was in, the call made, the money booked, the state it landed in, and the call made there. SARSA needs nothing else. The answer to the rung, in one line: read the logbook, and after each lived week nudge one cell toward what it actually paid. The rest of this section earns that sentence.

Maintain a table  $q(s, a)$  of *estimates*, current best guesses of  $Q^*(s, a)$ , one cell per state-action pair, twelve for the fleet, with every cell starting at 0, as in the visualization. After every lived week we will improve the single cell just used. The only question is how.

### Three bookkeepers, one bad week

Forget the future for a moment and ask the simplest version of the question: what does *one* week of a given pair pay on average, say (WORN, RUN)? This is the bare averaging problem of Section 7, now asked of a single cell. Nobody prints the odds, but the logbook shows what those weeks actually paid: +72, +72, +72, −208, +72, ... (an excerpt, mostly profits with the occasional breakdown that books the 72 profit minus the 280 repair). Picture three bookkeepers keeping the cell, each fed this same stream.

The *Overwriter* trusts only the latest week: the cell is whatever last week paid. Across the five weeks the cell reads 72, 72, 72, then −208, then 72 again. One breakdown and the cell screams that servicing a worn van loses money; the next ordinary week and it forgets the breakdown happened. Hysterical.

The *Filing cabinet* keeps the true running average of every week. Its cell reads 72, 72, 72, then  $(72 + 72 + 72 - 208)/4 = 2$  after the breakdown, then  $(72 + 72 + 72 - 208 + 72)/5 = 16$ . Calm and correct, the breakdown absorbed into a long record, one disaster among many good weeks. The price: it must store every card forever to recompute the average.

The *Nudger* keeps one number and, each week, moves it 15% of the way toward what the week paid. Its cell reads 72, 72, 72, then  $72 + 0.15(-208 - 72) = 30$  after the breakdown, then  $30 + 0.15(72 - 30) = 36.3$ . It tracks the filing cabinet, 30 then 36.3 against 2 then 16 (closer once a few more good weeks roll in), and it stores nothing but that single number. Calm, and no cabinet.

The Nudger is the rule we want, and it has a tidy form. Writing  $q(s_i, a_i)$  for the cell of the pair used in week  $i$ ,  $r_i$  for what the week paid, and  $\alpha$  for the fraction of the gap to close ( $\alpha$  is the Greek letter alpha, the *learning rate*, 0.15 here):

$$q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha[r_i - q(s_i, a_i)]. \quad (8.1)$$

In words: the arrow is an instruction, not an equation; compute the right side and replace the left side by it, moving the estimate a fraction  $\alpha$  of the way toward what the week actually paid. The bracket  $r_i - q(s_i, a_i)$  is the *surprise*: the gap between what the week paid and what

---

<sup>7</sup>Scene “Why DP doesn’t scale” of the interactive [visualization](#) prices this out.

the cell believed. This is precisely the Robbins-Monro nudge (7.1), applied to one cell, with the arriving sample being the week's cash  $r_i$  and the step held constant at  $\alpha$ . The moral of the three bookkeepers is the design rule for everything that follows: nudge, never overwrite, so one bad week cannot erase a year.

## The alpha knob

How big should the nudge be? The fraction  $\alpha$  is a knob, and the same five weeks settle very differently at three settings. At  $\alpha = 0.9$  the cell lurches almost all the way to each week: when the breakdown lands it drops to  $72 + 0.9(-208 - 72) = -180$ , and the next ordinary week yanks it back to  $-180 + 0.9(72 - (-180)) = 46.8$ . Whiplash. At  $\alpha = 0.05$  the cell barely moves: the breakdown nudges it only to 58, and the next week to 58.7, so slow it would take an age to learn anything. At  $\alpha = 0.15$  the cell lands at 30 then 36.3, the Nudger's calm middle from above: it registers the breakdown without being ruled by it. Small steady steps, never overwrite, is the lesson; 0.15 is the visualization's choice.

## The planner's Monday

Formula (8.1) is already on the manager's desk. A sales planner sets Monday's forecast at 100; the week sells 80; the miss is  $80 - 100 = -20$ ; and next Monday's forecast becomes  $100 + 0.15(-20) = 97$ , the old forecast nudged by a fraction of the miss. That update,

$$\text{new forecast} = \text{old forecast} + \alpha(\text{miss}),$$

is exactly (8.1) with the cell  $q(s_i, a_i)$  playing the old forecast and  $r_i - q(s_i, a_i)$  the miss. Its textbook name is *exponential smoothing*, the standard forecast-revision rule of demand planning. SARSA forecasts the value of a call the way a planner forecasts sales.

## The cash-only ledger lies

Rule (8.1) learns what a pair pays *this week in cash*, and that is not yet the number we are after. Look back at the crowned cells of Section 6. On the cash sheet, REPLACE pays  $-130$  every single week, the worst line anywhere, and SERVICE pays  $-50$ ; yet the scorecard  $Q^*$  already crowned REPLACE at SHAKY and FAILING (149.9) and SERVICE at WORN (226.1). A cash-only ledger, taking each call at its weekly money, would brand the two winning calls as losers.

What does the cash sheet miss? Two weeks can book the *same* cash and still leave you holding very different vans, one HEALTHY in the lot and one FAILING. Same cash, different futures. The next subsection is the resolution.

## A call also leaves you a van

A call does *two* things at once. It pays cash now, and it leaves you a van in some state to carry into next week, and that leftover van has worth. RUN from HEALTHY earns  $+95$  this week, but it ends with a more worn van and a thinner future (a breakdown leaves you FAILING). REPLACE costs 130 this week, but it hands you a brand-new HEALTHY van next Monday, a future full of profit. The cash sheet sees only the first half of each call. REPLACE loses money on the week, and yet the HEALTHY van it leaves is worth far more than 130 down the road; that, not its weekly cash, is why the scorecard crowns it. So a call's true worth is its cash *plus the van it leaves you*, and the question that forces the rest of the derivation is how to put a number on that leftover van.

## Grade the week like a trade-in

One line of the logbook is  $s_i, a_i, r_i, s_{i+1}, a_{i+1}$ : the state, the call, the cash, the state you land in, and the call you make there. (Read down the five letters: that spells *SARSA*.) End the week and you are holding a new pair, the landing state  $s_{i+1}$  and the call  $a_{i+1}$  you will play in it. Grade it like a trade-in: its value is  $q(s_{i+1}, a_{i+1})$ , read from your own table, discounted by  $\gamma$  because it is a future and not cash in hand, exactly the discounting of the return in (4.2). The week's worth is then the cash plus that trade-in,

$$\text{the week's worth} = r_i + \gamma q(s_{i+1}, a_{i+1}),$$

and this, not  $r_i$  alone, is the “actual” the nudge should aim at. We have upgraded the target from cash alone to cash plus the van you keep. This is the recurrence  $G_i = r_i + \gamma G_{i+1}$  of (4.3) again, with the leftover return appraised from the table.

## Price it from your own book

There is a fair objection: the trade-in is priced with  $q$ , the very table we are still learning. Are we not using a number we do not yet know to learn that number? A used-vehicle dealer faces the same loop and shrugs: this month's price book is imperfect and still being revised, yet the dealer prices today's trade-in from it anyway, and nobody refuses to value a car because the book is not final. SARSA does the same. The cell  $q(s_{i+1}, a_{i+1})$  stands in for the unknown  $Q^*(s_{i+1}, a_{i+1})$ , the same move as plugging a forecasted cash flow into an NPV calculation. The book is wrong today, and that is fine: it starts at 0 everywhere and climbs toward the worth of the playbook you are running, every nudge a little less wrong than the last. Making this bootstrapping fully rigorous took decades; the averaging guarantee underneath it is the Robbins-Monro result of Section 7.

## Write the line yourself

Three pieces are now on the table, each one already named: the cash  $r_i$ ; the value of where you land,  $\gamma q(s_{i+1}, a_{i+1})$ ; and what you believed,  $q(s_i, a_i)$ . Drop the upgraded target into the smoothing rule (8.1) in place of the cash-only actual, and the full SARSA update writes itself:

$$q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha [r_i + \gamma q(s_{i+1}, a_{i+1}) - q(s_i, a_i)]. \quad (8.2)$$

In words: the same forecast revision as before, only the “actual” grew up; it is now this week's money plus the discounted value of where the week left you. The bracket is the surprise again (the planner's miss), but a richer one: the gap between what this week's evidence now says the pair is worth and what the table currently believes. Positive means the week beat the table's belief; negative means it fell short. You did not memorize this update; you wrote it from three pieces you had already built.

## Bellman becomes SARSA

Set the update (8.2) beside the Bellman equation (6.1) and the kinship is exact, in four steps. *First*, start from Bellman,  $Q^*(s_i, a_i) = \mathbb{E}[r_i + \gamma Q^*(s_{i+1}, a_{i+1})]$ , an average over all the weeks that could happen. *Second*, erase the average and keep the single week that actually arrived:  $Q^*(s_i, a_i) \approx r_i + \gamma Q^*(s_{i+1}, a_{i+1})$ . *Third*, erase the unknown truth  $Q^*$  and look the value up in your own book  $q$ :  $q(s_i, a_i) \approx r_i + \gamma q(s_{i+1}, a_{i+1})$ . *Fourth*, do not jump to that target, nudge toward it by a fraction  $\alpha$ , which is exactly (8.2). SARSA is the Bellman equation with the expectation replaced by an arriving average and  $Q^*$  replaced by the current guess  $q$ . (Definition 4.3 defined the expectation as an average over many runs in the first place, which is why an arriving average is a faithful stand-in for it; the averaging that licenses the swap is the Robbins-Monro result of Section 7.)

## Which call do we try while still learning?

The  $\varepsilon$ -greedy rule of Section 5, now used to drive a live van. At state  $s$ , mostly play the call whose  $q(s, a)$  is largest (exploit), but in a fraction  $\varepsilon = 0.15$  of the weeks, held constant like  $\alpha$ , play a random call instead (explore), because a cell that is never visited is never improved. One consequence matters for the update:  $a_{i+1}$  in (8.2) is the call this playbook *actually* makes next, exploratory weeks included, never by definition the largest-valued call. So SARSA learns the value of what it really does, which is the subtlety its cousin Q-learning, below, turns on.

## One week, one nudge

To see a single update land, feed in week one of Example 3.4 with the table still all zeros. The log-book line is (HEALTHY, RUN, +95, WORN, SERVICE); the evidence is  $r_i + \gamma q(\text{WORN, SERVICE}) = 95 + 0.9 \cdot 0 = 95$ ; the surprise is  $95 - 0 = 95$ ; and the cell  $q(\text{HEALTHY, RUN})$  moves from 0 to  $0 + 0.15 \cdot 95 = 14.25$ . One cell moves; the other eleven stay put. Run this week after week on one endless stream of lived experience, drive, get billed, nudge one cell, and the table drifts toward the worth of the playbook it is actually following. In the visualization the same three bands as in the scorecard of Section 6 (RUN when HEALTHY, SERVICE when WORN, REPLACE when SHAKY or FAILING) emerge within a few hundred weeks, the odds never shown.<sup>8</sup> One honest caveat, in two parts. First, because  $\varepsilon$  stays above 0, the playbook keeps running the occasional deliberately-imperfect experiment, so the worth it settles at is the worth of *that* slightly-imperfect playbook, which sits below the  $Q^*$  values rather than reaching them. Second, holding  $\alpha$  constant leaves the twelve numbers jittering around that target instead of locking onto it. But which call tops each row, and that is all the playbook reads, comes out right.

## SARSA, all together

Every thread of these notes now meets in a single loop, repeated week after week:

1. You are in a state, the van's wear level.
2. Choose a call by the  $\varepsilon$ -greedy rule of Section 5: usually the best you currently know (exploit), now and then a random one (explore).
3. Act; collect the week's reward  $r_i$ ; the van lands in a new state  $s_{i+1}$ , where the same rule picks the next call  $a_{i+1}$ .
4. Nudge your estimate of the call you just made a fraction  $\alpha$  of the way toward this week's evidence  $r_i + \gamma q(s_{i+1}, a_{i+1})$ : the Robbins-Monro nudge of (7.1) aimed at the Bellman target of (6.1).
5. Repeat. The estimates settle, and the best call per state emerges.

Four earlier rungs are bolted together here: the MDP supplies the states and calls (step 1), explore-exploit governs the choice (step 2), the Bellman equation supplies the target (step 4), and Robbins-Monro supplies the nudge (step 4). That is the whole of SARSA.

## SARSA's cousin: Q-learning

The caveat just above, that SARSA settles a little below  $Q^*$ , traces to one design choice in the target. The evidence  $r_i + \gamma q(s_{i+1}, a_{i+1})$  uses  $a_{i+1}$ , the call the playbook will *actually* make next, which by the  $\varepsilon$ -greedy rule is sometimes a deliberately random experiment rather than the call the table favors. So SARSA samples the Bellman equation of *the very policy it is*

---

<sup>8</sup>Scene "SARSA: learn from the logbook" of the interactive [visualization](#): watch one week nudge one cell.

following, exploratory weeks and all, and converges to the worth of that slightly-imperfect playbook. SARSA learns the road it actually walks; this is called an *on-policy* method.

Change one word in the target and you get its famous cousin. *Q-learning* aims not at the call it will really make next but at the *best* call available next, whatever the table rates highest there:

$$q(s_i, a_i) \leftarrow q(s_i, a_i) + \alpha [r_i + \gamma (\text{value of the best call at } s_{i+1}) - q(s_i, a_i)].$$

Because the call it learns about need not be the call it takes, Q-learning chases  $Q^*$  itself even while it wanders off to explore; it is therefore called *off-policy*. The nudge is identical and only the next-call in the target changed: SARSA asks “what will I really do next?” and Q-learning asks “what is the best I could do next?”. On the fleet both settle on the same three winning bands; they differ only in what they make of the occasional experimental week.<sup>9</sup>

## Closing

These notes modeled sequential decision problems as Markov decision processes, let policies generate random trajectories, scored each trajectory by its cumulative reward, and averaged that score to obtain something maximizable. They framed the explore-exploit tension every learner faces, characterized the best policy through the  $Q$ -function and the Bellman equation, and saw in the Robbins-Monro algorithm how to learn an average from its samples. When the odds are unknown, SARSA bolts those last two together, learning the same scorecard from the logbook alone, one nudge per lived week, with Q-learning its off-policy cousin. The fastest way to make all of this concrete is to play: open <https://eth-ainit-fs26.github.io/rl-arcade/repair-or-replace/>, run the fleet for a quarter, drag the  $\gamma$  slider, and watch the  $Q^*$  scorecard fill in, scene by scene, in the same order as these notes.

## A The running average

Section 7 gave the general Robbins-Monro principle; here is the elementary, two-line derivation of the same nudge that it promised, and a note on what the constant step  $\alpha$  buys in place of the shrinking  $1/n$ . Let  $A_n$  be the average of the first  $n$  observations  $x_1, \dots, x_n$ , that is,  $A_n = (x_1 + \dots + x_n)/n$ . Then

$$n A_n = x_1 + \dots + x_n = (n-1) A_{n-1} + x_n.$$

In words:  $n$  times the average is the total of all  $n$  observations, and that total is the total of the first  $n-1$  of them, which is  $(n-1) A_{n-1}$  (average times count), plus the newest one. Dividing by  $n$  and rearranging,

$$A_n = A_{n-1} + \frac{1}{n} (x_n - A_{n-1}).$$

In words: each new observation moves the average by  $1/n$  of the surprise, the gap between the newcomer and the average so far. A two-number check: the average of 10 and 16 is 13; a third observation of 22 moves it to  $13 + \frac{1}{3} (22 - 13) = 16$ , which is indeed the average of 10, 16, and 22.

Now replace the shrinking  $1/n$  by a small constant  $\alpha$ . The result is an average that gently favors recent observations: each old observation’s influence shrinks by a factor  $1 - \alpha$  with every new week. That is the right choice here, because the early evidence was measured against a

---

<sup>9</sup>The visualization’s SARSA scene follows the road it walks; Q-learning is its standard counterpart in any reinforcement-learning course.

poor table  $q$  and should fade. With that one substitution, the running-average recipe *is* the SARSA update (8.2): the rule is nothing more exotic than a running average of the weekly evidence.